



# CIRCUITOS DIGITALES Y MICROCONTROLADORES 2025



Facultad de Ingeniería  
UNLP

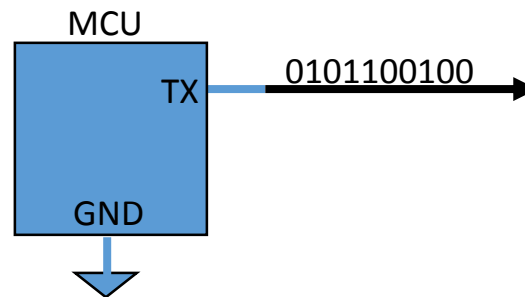
Periférico de comunicación serie  
asincrónico - UART

Ing. José Juárez

Video de la clase dictada por Ing. José Juárez  
para el curso Ingeniería en Automatización y Control Industrial -UNQ:  
<https://www.youtube.com/watch?v=AdHtVmxxDFM>

# Conceptos básicos de una comunicación serie

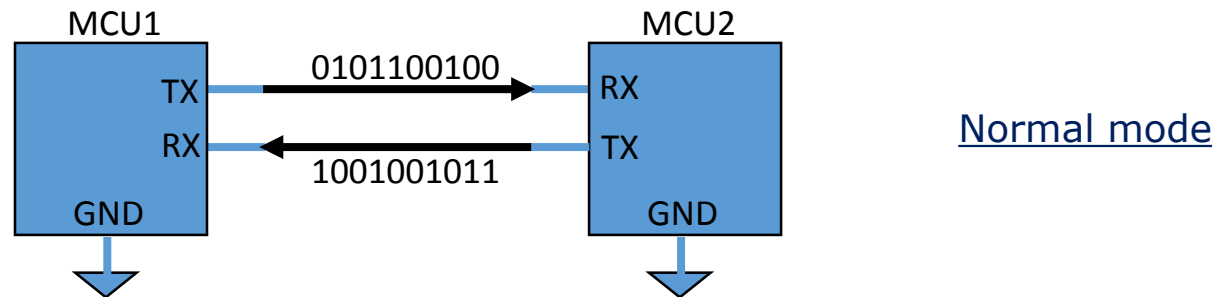
- Transmisión serie: Los datos se envían en paquetes de varios bits, UN BIT A LA VEZ, por el mismo canal de comunicación.



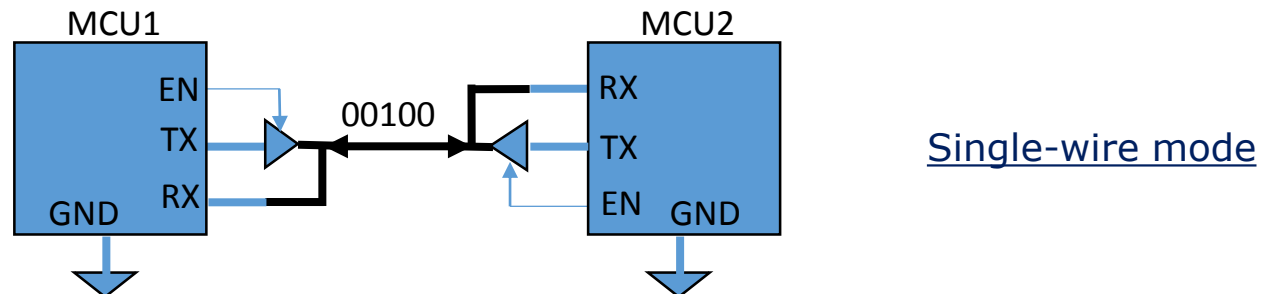
- Tiempo de Bit ( $T_b$ ): es el tiempo de duración de un bit
- Tasa de Transferencia ( $1/T_b$ ): es el número de bits por unidad de tiempo [bps]. También se la denomina baud-rate (tasa de símbolos).
- Overhead: son bits o bytes que se “agregan” al dato para hacer más confiable una transmisión. Por ejemplo: bit de paridad o bytes de cksum.
- Bandwidth o Throughput: es el número total de bits de información por unidad de tiempo, sin tener en cuenta el Overhead.

# Conceptos básicos de una comunicación serie

- Full- dúplex: transmisión y recepción en ambos sentidos simultáneamente. Requiere de hardware separados y dos canales de comunicación.



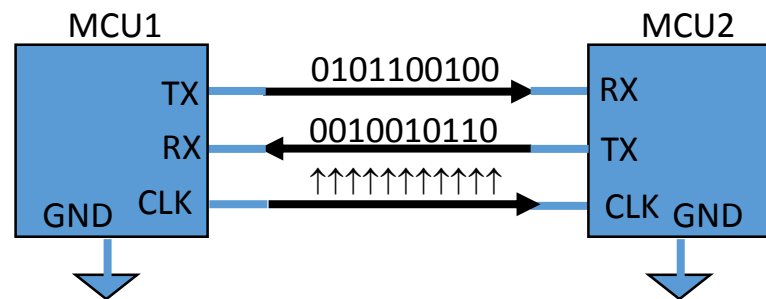
- Half Duplex: la comunicación es bidireccional pero no simultánea sobre un solo canal (un problema fundamental de estos sistemas es el manejo de Colisiones)



- Simplex: la comunicación es en un solo sentido en un solo canal.

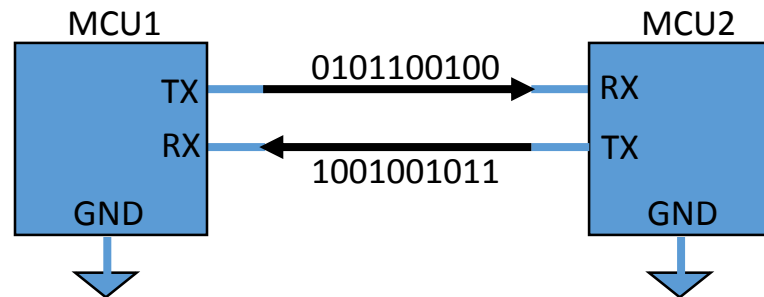
# Conceptos básicos de una comunicación serie

- Sistema sincrónico: el receptor y el transmisor se deben sincronizar a una tasa de transferencia dada, empleando un reloj común a ambos (orientado a la transferencia de bloques de datos).
  - La ventaja es que se pueden utilizar tasas de transferencia más altas.
  - La desventaja es que requiere un conductor adicional para transmitir la señal de reloj

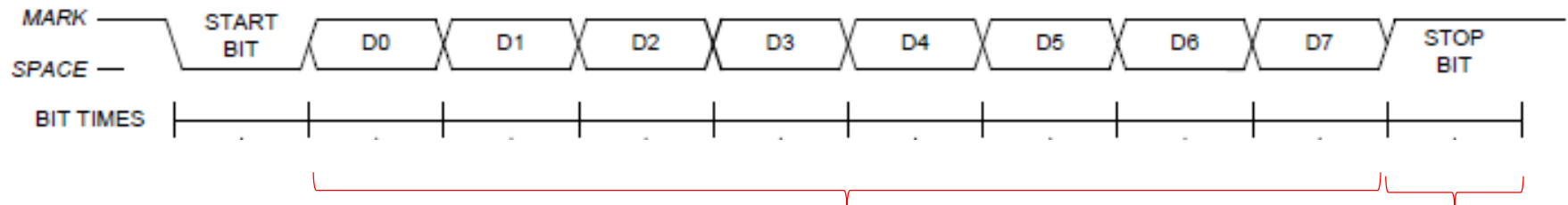


# Conceptos básicos de una comunicación serie

- Asincrónico: El TX y RX no están sincronizados por un reloj común, si no que la tasa de transferencia se supone conocida y la trama de datos contiene un bit de comienzo y otro de fin para sincronizar el RX y decodificar los datos (orientado a transferencia de caracteres)



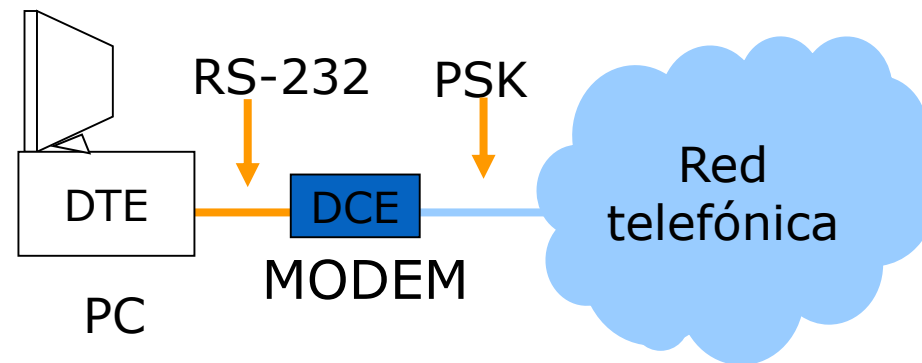
- Un ejemplo muy utilizado del modo asincrónico es el formato **8N1**



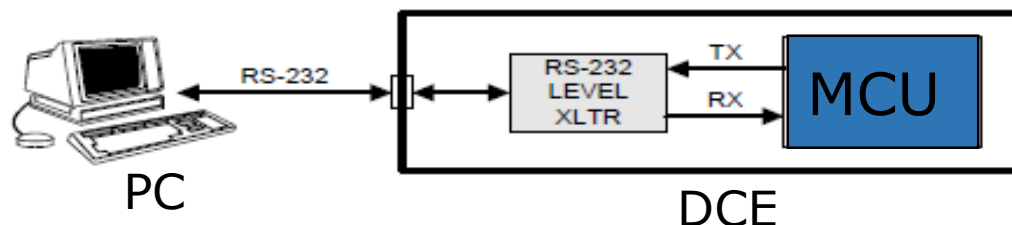
**8 bits de datos, No paridad, 1 bit de stop**

# El estándar RS-232 (Recommended Standard 232)

- Propuesto por la EIA (Electronics Industry Association) en 1960, define el método más popular para interconectar DTEs (Data Terminal Equipment) y DCEs (Data Communication Equipment), por ejemplo, conexión entre un PC y un MODEM de datos o una impresora.



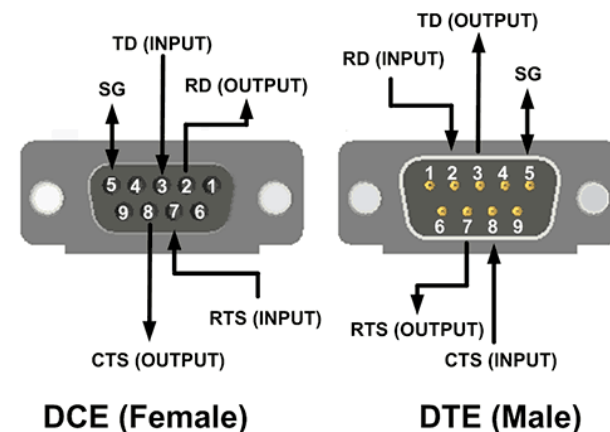
- En realidad, un Modem o una Impresora es un Sistema Embebido:



# RS-232: Características Eléctricas

- Especifica tasas de transferencia no mayor a 20Kbps para distancias de interconexión de 15m
- Y tasas de transferencia hasta 115kbps para distancias más cortas.
- La interfaz puede operar en modo asincrónico, síncrono, full-dúplex, half-dúplex o simplex.
- Define un 1 lógico (Mark) como una tensión entre -3 y -15V
- Un 0 lógico (Space) como una tensión entre +3 y +15V
- La región (-3,3) no esta definida
- Especificación de señales desde el lado DTE (PC):

| Señal               |     | DB-25 | DE-9 (DB-9, TIA-574) |
|---------------------|-----|-------|----------------------|
| Common Ground       | G   | 7     | 5                    |
| Transmitted Data    | TD  | 2     | 3                    |
| Received Data       | RD  | 3     | 2                    |
| Data Terminal Ready | DTR | 20    | 4                    |
| Data Set Ready      | DSR | 6     | 6                    |
| Request To Send     | RTS | 4     | 7                    |
| Clear To Send       | CTS | 5     | 8                    |
| Carrier Detect      | DCD | 8     | 1                    |
| Ring Indicator      | RI  | 22    | 9                    |

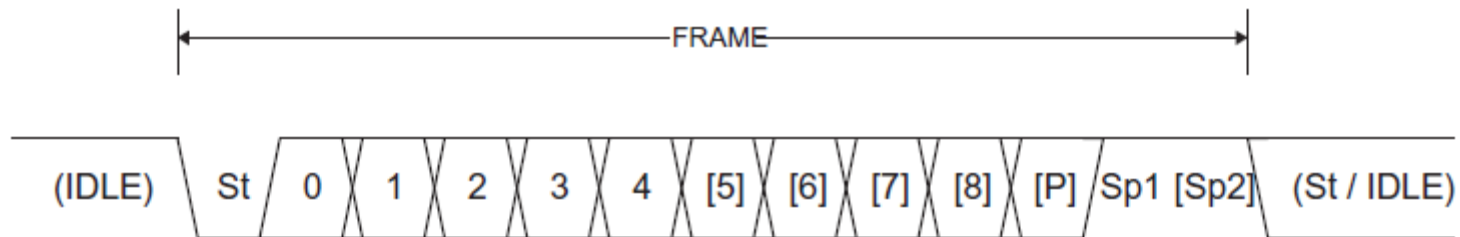


Ejemplo de secuencia de inicialización de un MODEM:

- 1) Power On
- 2) PC activa DTR y el modem responde con DSR
- 3) El modem establece el enlace con modem remoto a través de la línea telefónica
- 4) El modem le avisa que se estableció conexión activando DCD
- 5) Cuando la PC desea transmitir activa RTS
- 6) El modem reconoce el pedido activando CTS

# RS-232: Formato de trama

- Cuando no se envían datos la señal se debe mantener en estado de marca (un uno lógico, conocido también como *RS-232 idle state*).



- El comienzo de flujo de datos se reconoce porque la señal pasa de “marca” a “espacio”. Esta sincronización entre emisor y receptor generalmente se implementa como el bit de arranque.
- RS-232 NO especifica como representar caracteres (7 u 8 bits es la forma más común, pero podrían ser 5, 6 ó 9).
- bit de paridad (que es opcional, depende de la implementación) :
  - No Parity (sin paridad)
  - Even Parity (paridad “par”): el bit de paridad es uno (1) para que haya una cantidad par de unos.
  - Odd Parity (paridad “impar”): el bit de paridad es uno (1) para que haya una cantidad impar de unos.
- Después del bit de paridad (si lo hay) continúan los bits de parada (stop bits) que pueden ser 1, 1.5 o 2 dependiendo la implementación.
- La configuración más común es 8N1 pero podría ser cualquier combinación 5N2, 8E1, 7O2, etc



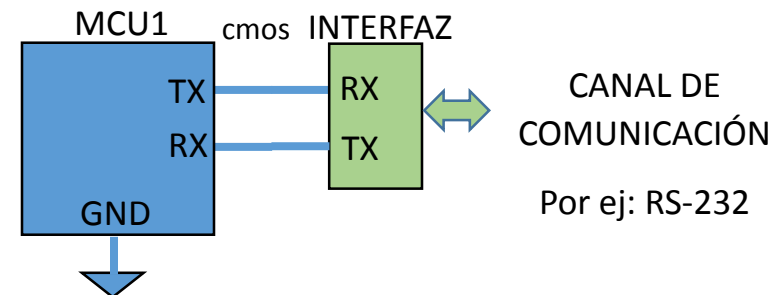
# RS-232: Control de flujo

- En RS-232 el control de flujo se puede hacer de dos maneras: por hardware (RTS/CTS) o por software (Xon/Xoff).
- Por Software: El carácter Xoff (ASCII 19) es utilizado por el receptor para decir que su buffer está lleno y el emisor debe esperar. Cuando vuelva a tener espacio en el buffer, enviará el carácter Xon (ASCII 17), diciéndole que puede volver a transmitir.
- Por hardware: a diferencia del anterior, este tipo de control de flujo requiere que entre la PC y el MODEM se conecten dos hilos más: RTS y CTS. Cuando el buffer del receptor se llena desactiva la señal CTS, la PC debe esperar . Cuando vuelva a tener espacio en el buffer, activa nuevamente el CTS para decir que está nuevamente listo.
- Los microcontroladores LPC1769 (ARM Cortex M3) poseen una UART con full Modem control.
- En microcontroladores AVR las señales CTS y RTS pueden implementarse con terminales I/O
- El módulo adaptador UART/USB con chip FTDI posee CTS y RTS

# UART (Universal Asynchronous Receiver Transmitter)

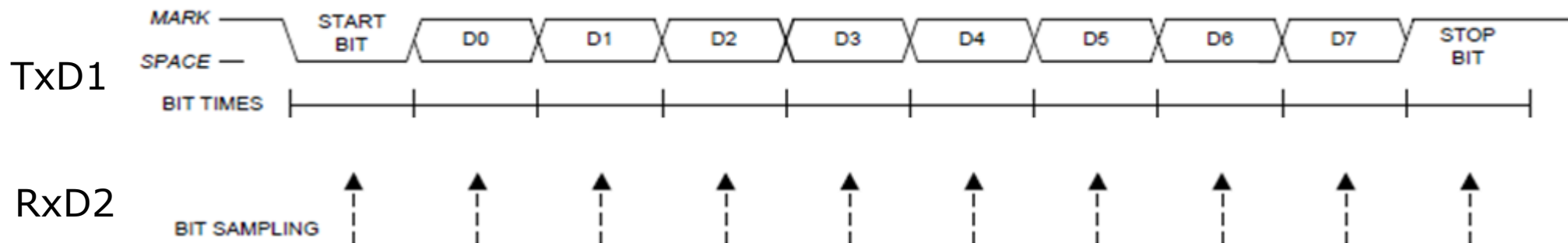
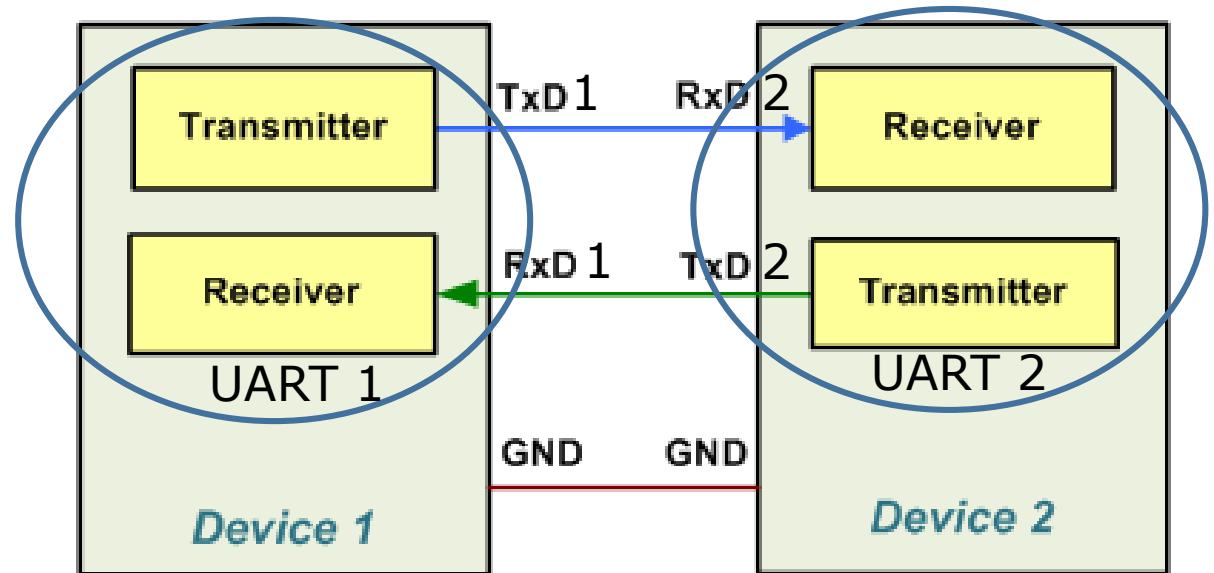
- Es el periférico integrado en un MCU que permite la comunicación serie asincrónica con otros dispositivos como una PC, modems u otros MCU.
- Con un MCU se puede utilizar tasas estándar de transmisión desde 1200 bps hasta 38400 bps o incluso 115200 bps, dependiendo de la frecuencia de operación del mismo.
- Algunos microcontroladores incorporan un periférico USART (Universal Synchronous Asynchronous Receiver Transmitter) capaz de transmitir el reloj para aumentar al máximo la tasa de transmisión.
- NXP (Freescale) utiliza el término SCI (Serial Communication Interface) para referirse a su dispositivo UART compatible.
- Un periférico UART trabaja con niveles TTL/CMOS

Conectando un dispositivo de interfaz adecuado al MCU, podemos proporcionar al sistema embebido de "conectividad"



# Comunicación entre dos dispositivos con UART

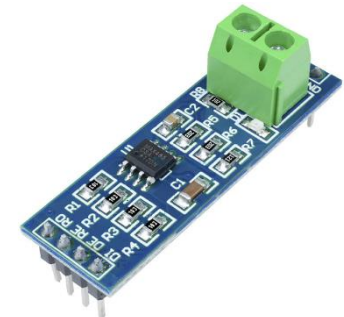
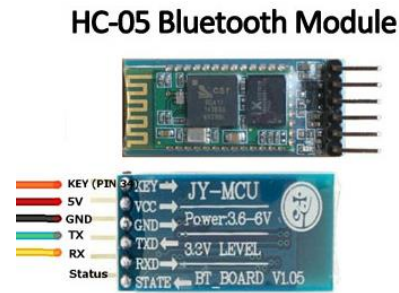
- Serial
- Asynchronous
- Full duplex
- 8N1



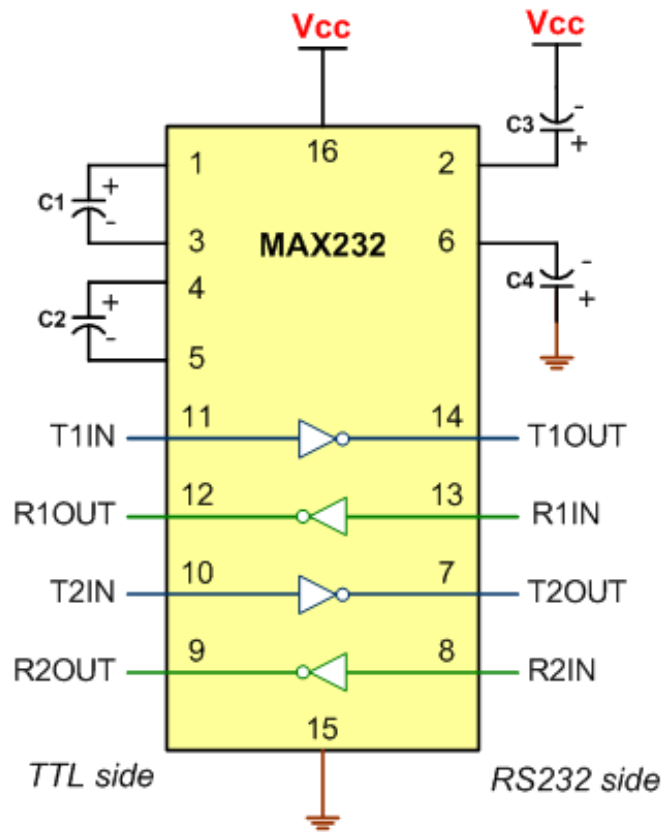
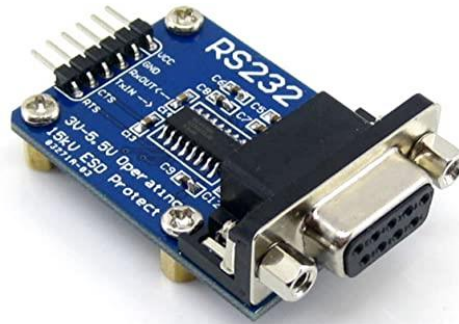
De la misma manera funcionarán TxD2 y RxD1

# UART (Universal Asynchronous Receiver Transmitter)

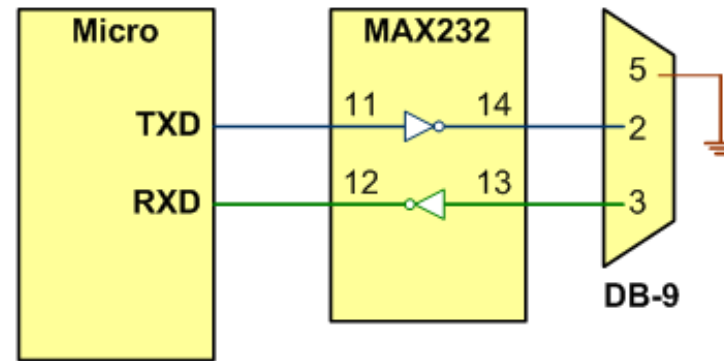
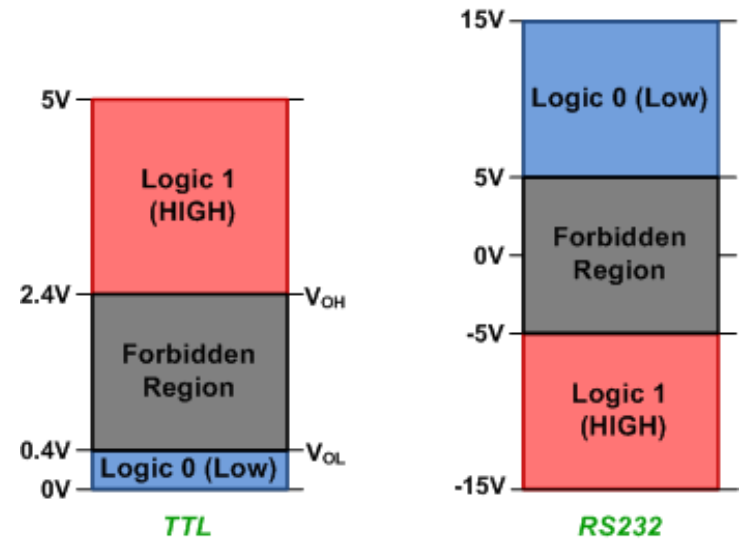
- módulos Bluetooth
- módulos GPS
- módulos GPRS,
- módulos WIFI,
- módulos RF,
- módulos RS232
- módulos RS485
- módulos CAN
- módulos USB



# RS-232



(a) Inside MAX232

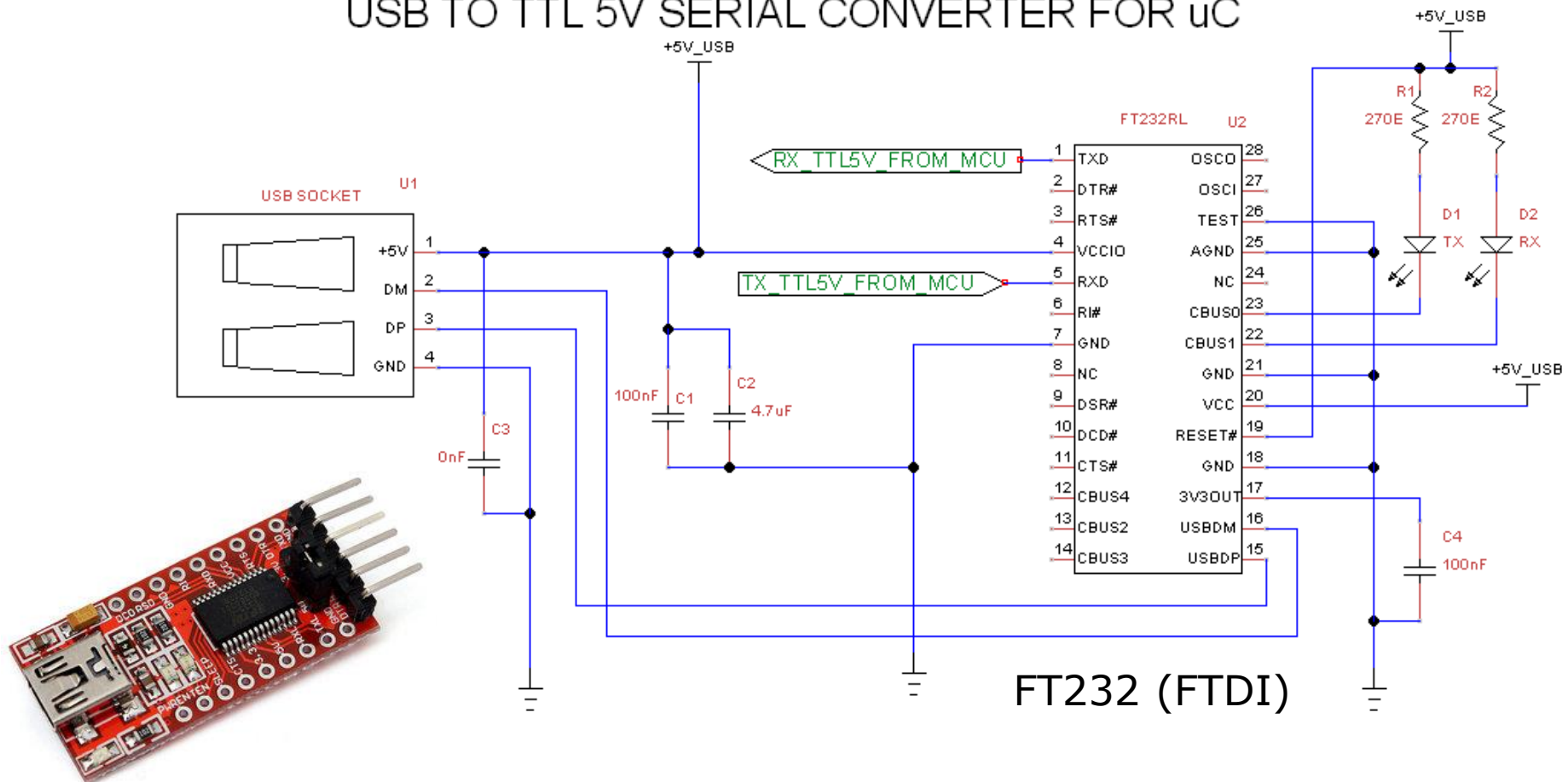


(b) MAX232 Connection to the Microcontroller / Microprocessor

# Implementación de comunicación por USB

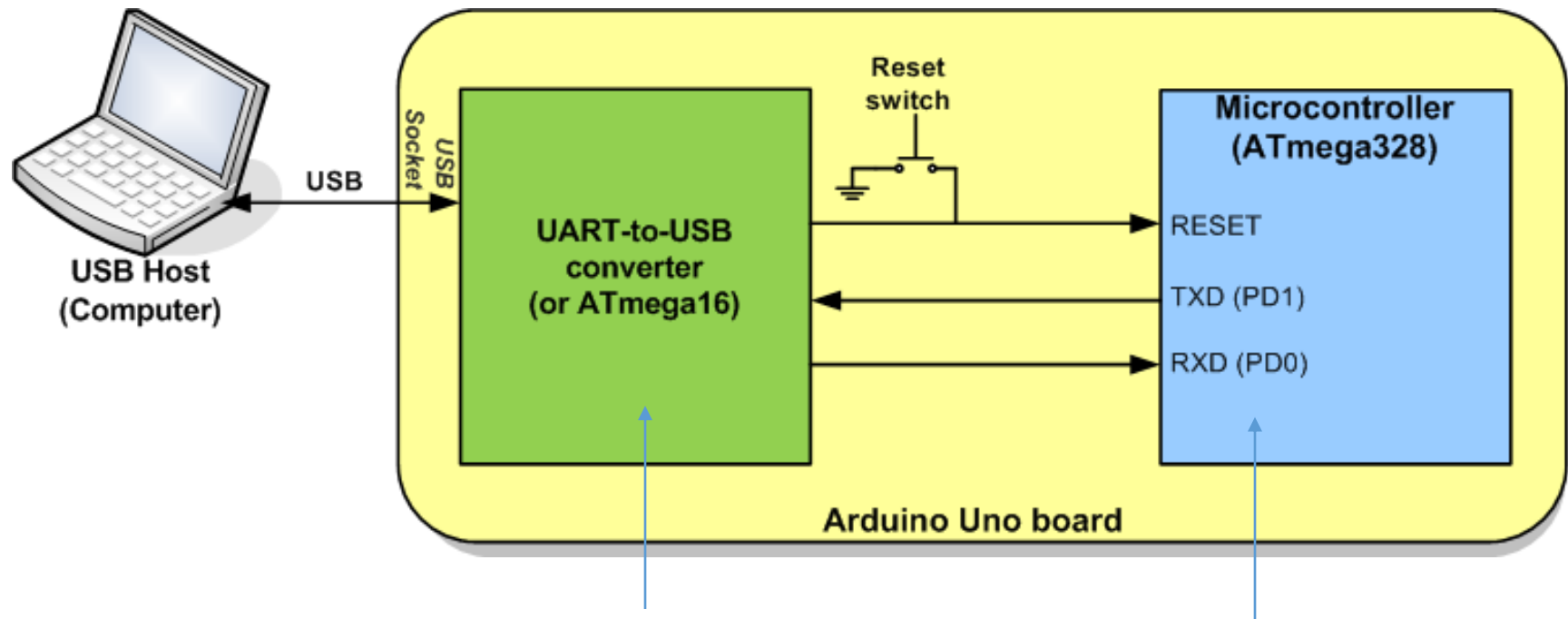
Diagrama eléctrico de Conexión a interfaz USB (clase CDC)

## USB TO TTL 5V SERIAL CONVERTER FOR uC



# Conversión USB a Serie en Arduino

- [USB CDC \(Communications Device Class\)](#)

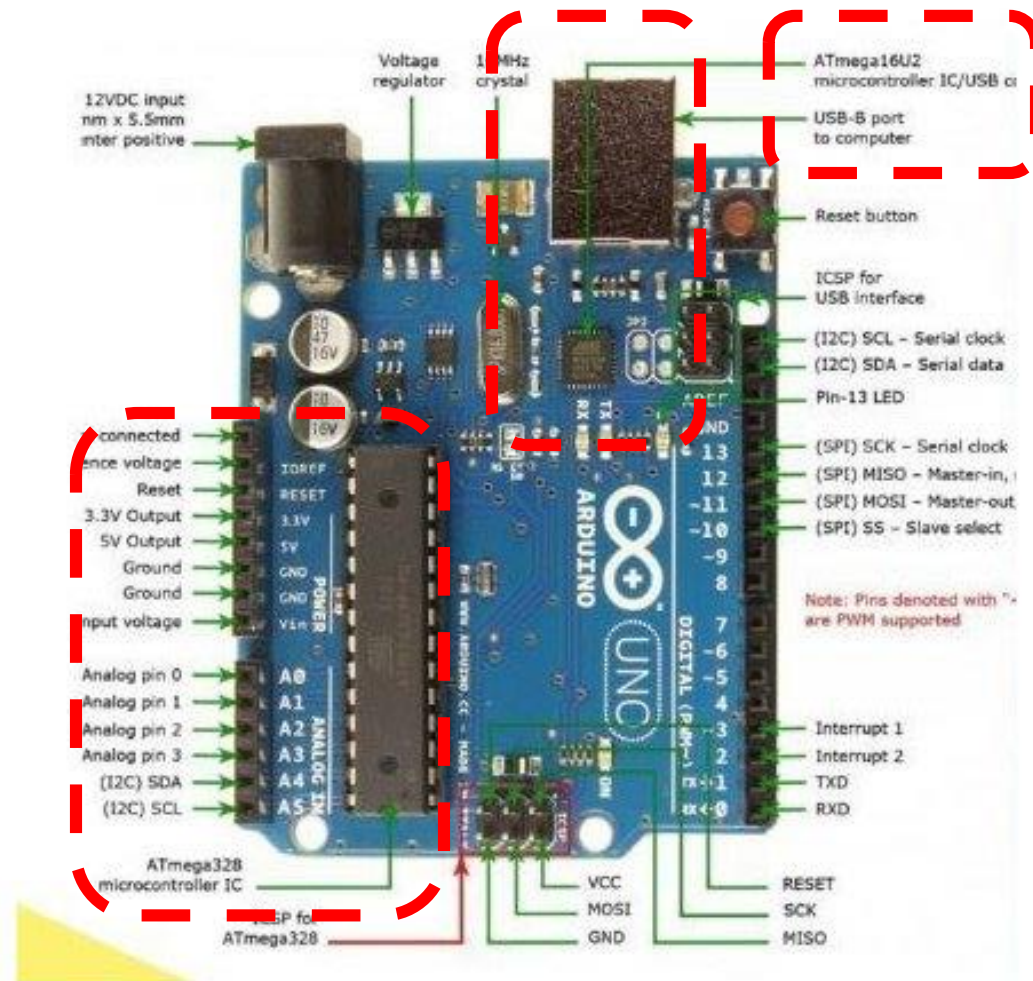


*Utiliza otro MCU con periférico USB  
para realizar la conversión de  
protocolos*

*El mCU Atmega328p posee 1 sola  
UART denominada **USART0***



# USB to serial converter en Arduino

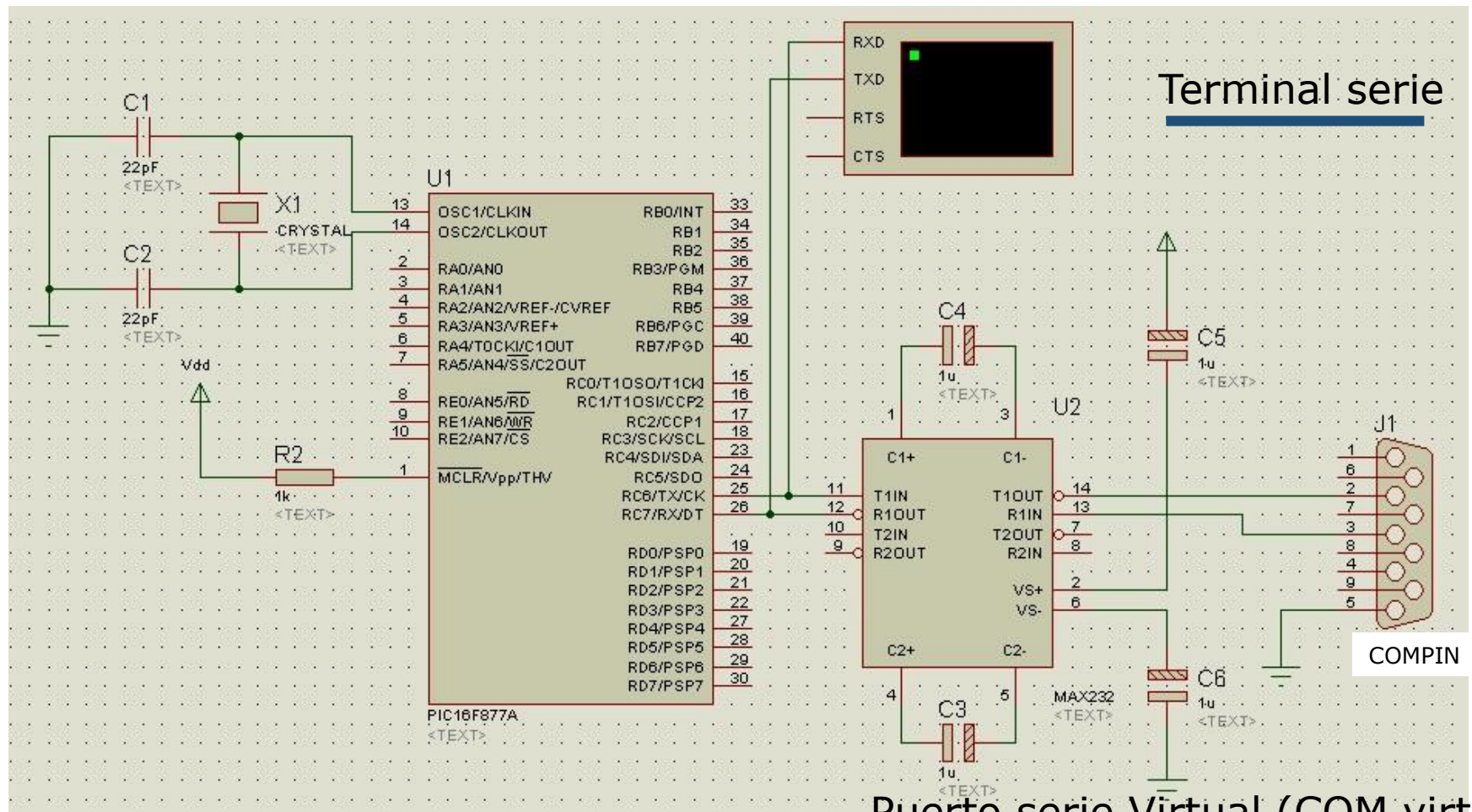


[How USB works](#)



# Simulación de comunicación serie en Proteus

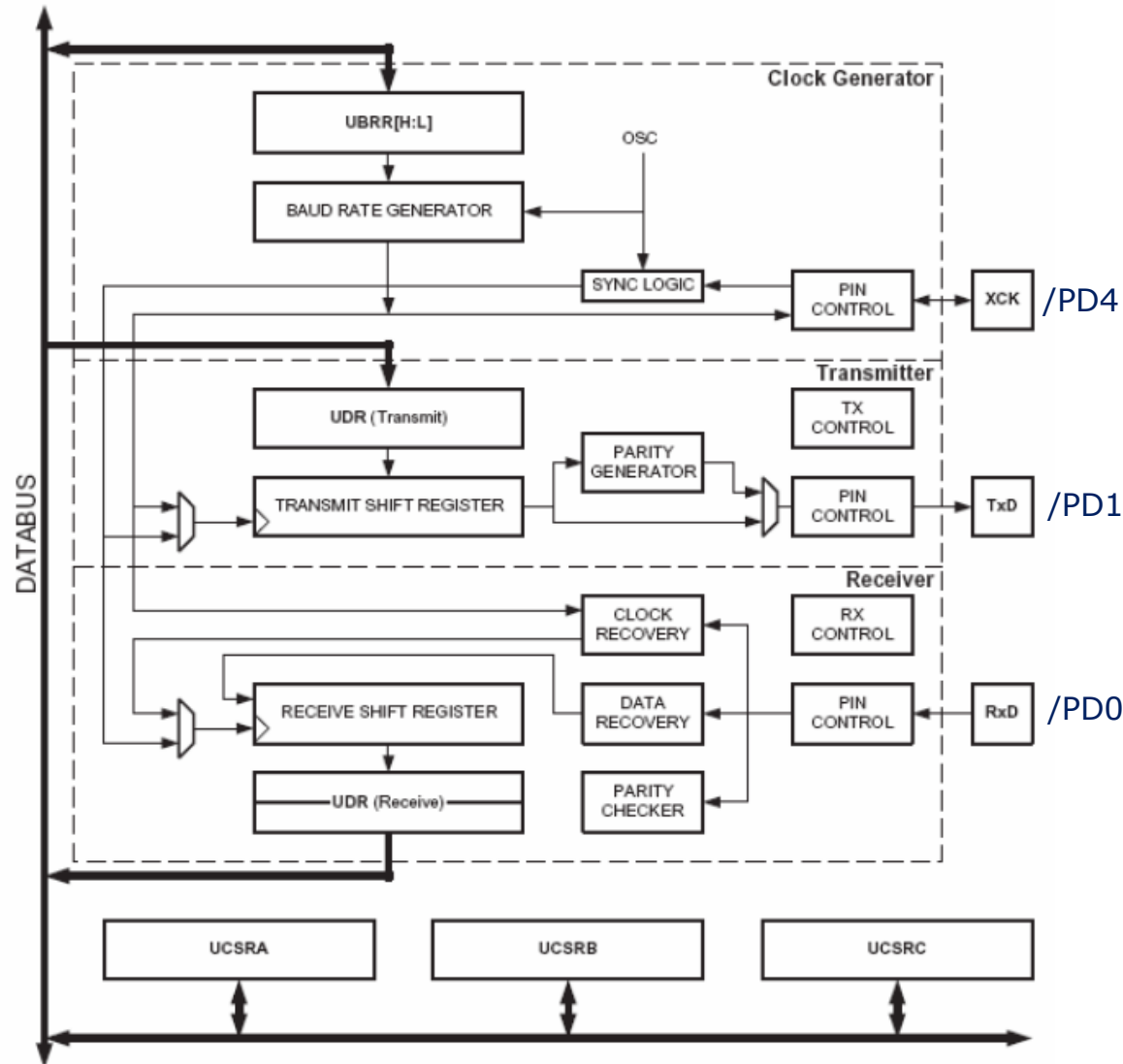
- Se dispone de dos opciones:



Puerto serie Virtual (COM virtual)

# Periférico serie en AVR: USART

(Universal Synchronous and Aynchronous Receiver Transmitter)

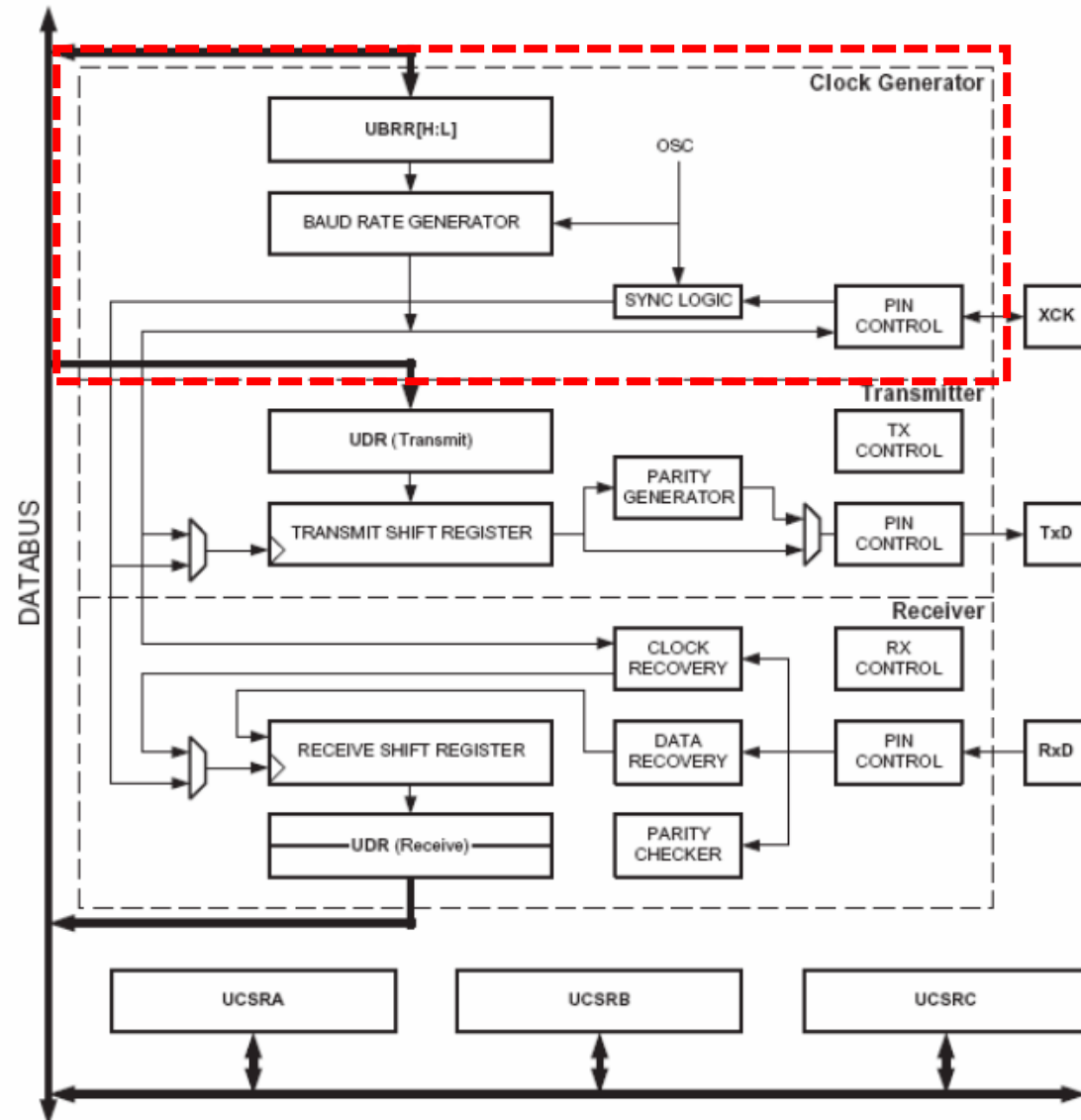


|                          |    |    |      |
|--------------------------|----|----|------|
| (PCINT14/RESET) PC6      | 1  | 28 | PC5  |
| (PCINT16/RXD) PD0        | 2  | 27 | PC4  |
| (PCINT17/TXD) PD1        | 3  | 26 | PC3  |
| (PCINT18/INT0) PD2       | 4  | 25 | PC2  |
| (PCINT19/OC2B/INT1) PD3  | 5  | 24 | PC1  |
| (PCINT20/XCK/T0) PD4     | 6  | 23 | PC0  |
| VCC                      | 7  | 22 | GND  |
| GND                      | 8  | 21 | AREF |
| (PCINT6/XTAL1/TOSC1) PB6 | 9  | 20 | AVCC |
| (PCINT7/XTAL2/TOSC2) PB7 | 10 | 19 | PB5  |
| (PCINT21/OC0B/T1) PD5    | 11 | 18 | PB4  |
| (PCINT22/OC0A/AIN0) PD6  | 12 | 17 | PB3  |
| (PCINT23/AIN1) PD7       | 13 | 16 | PB2  |
| (PCINT0/CLKO/ICP1) PB0   | 14 | 15 | PB1  |

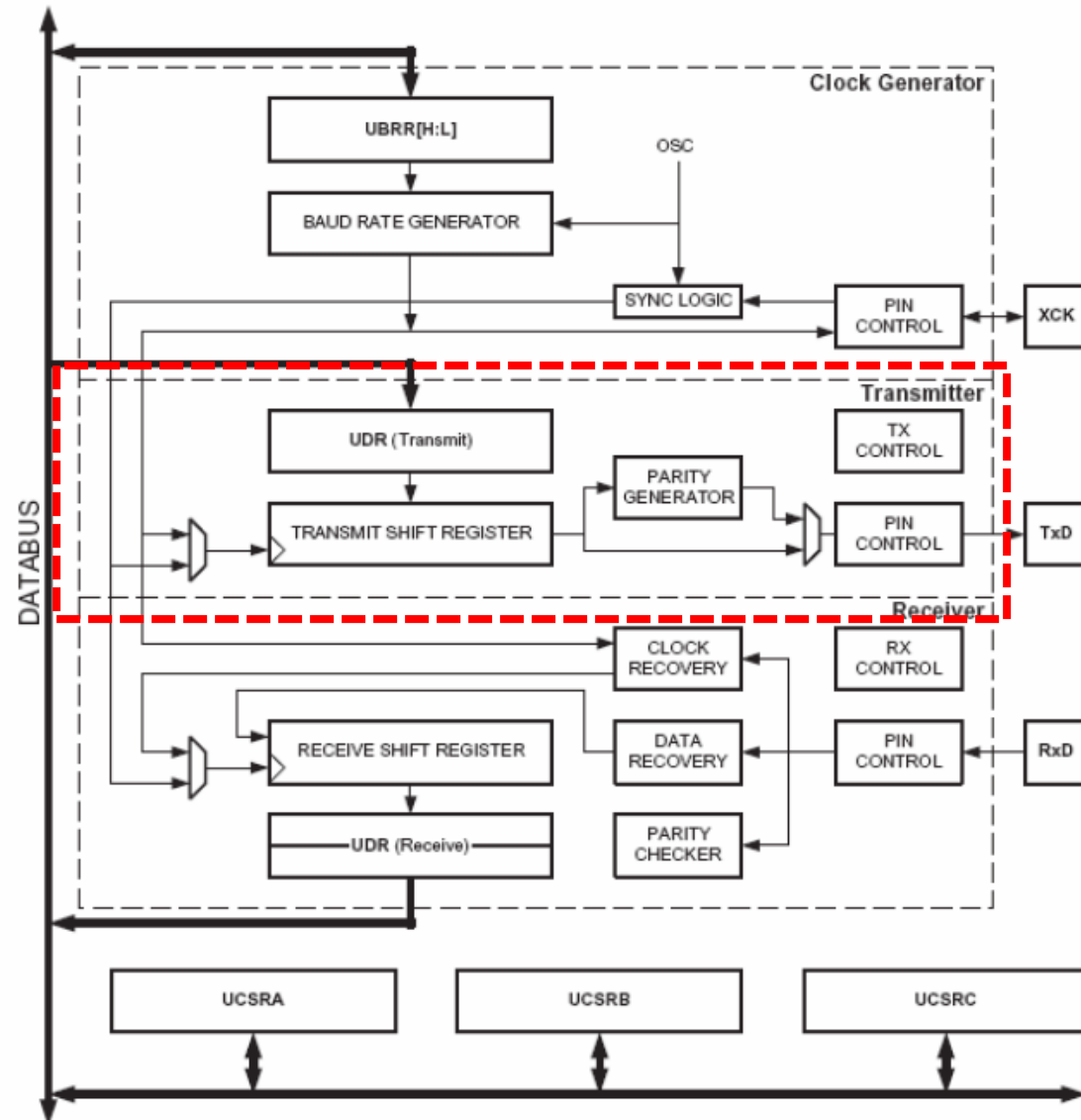
Algunas aplicaciones requieren de comunicación serie con múltiples dispositivos.

En el caso que el MCU no tenga disponible otra UART, se puede implementar uart por software. Se lo denomina *bit banging UART*.

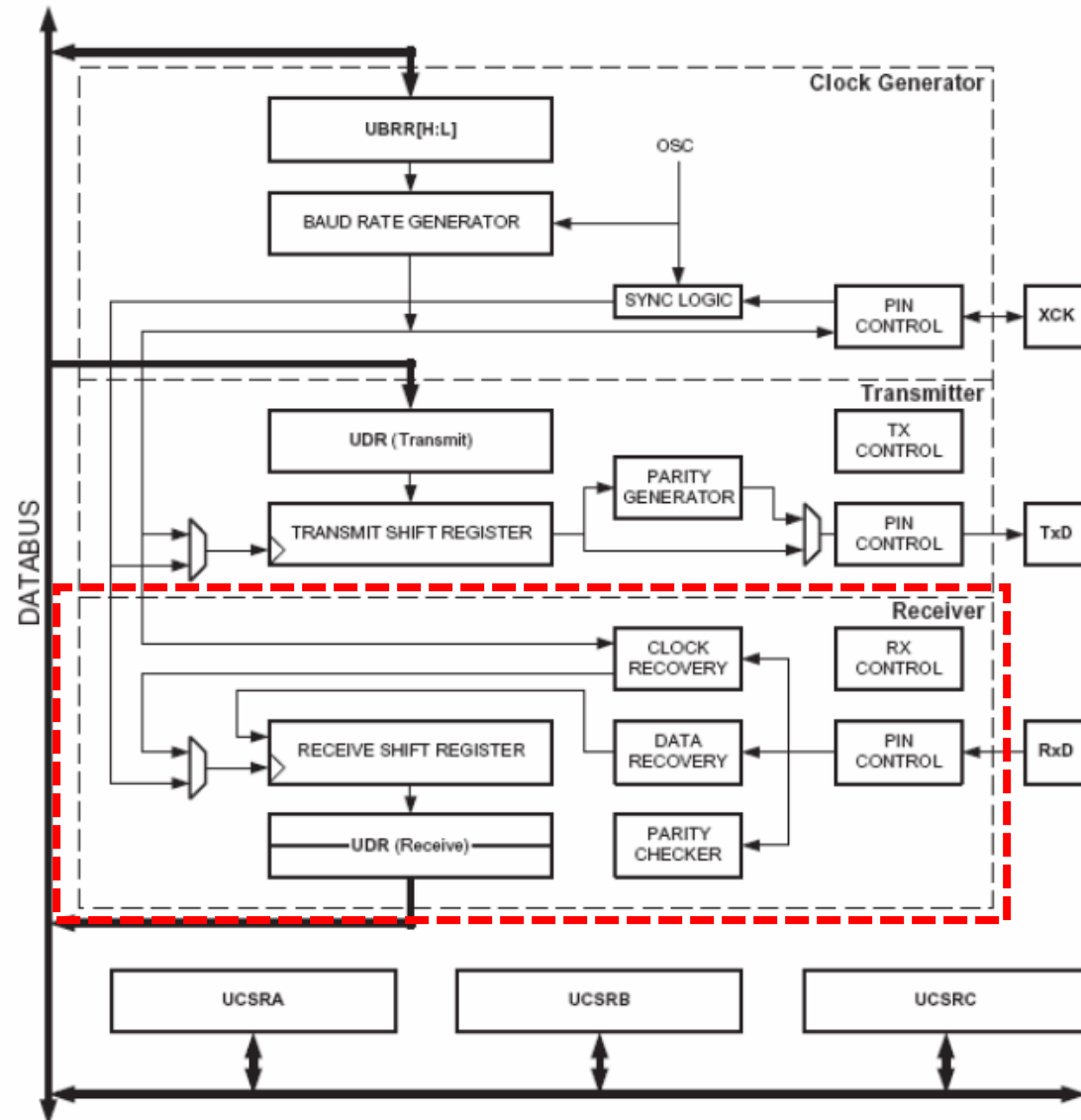
# Periférico serie en AVR: Generador de reloj



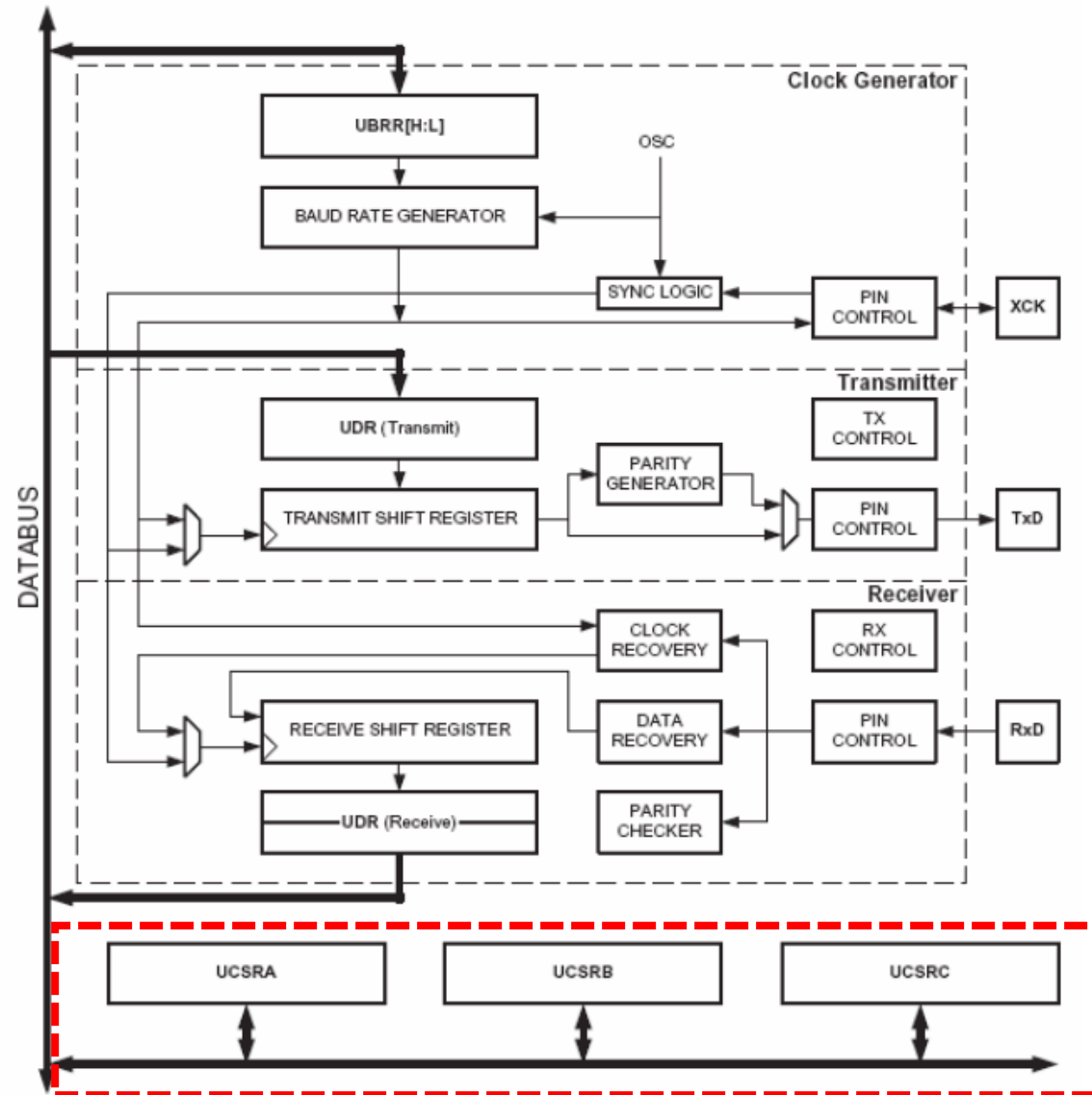
# Periférico serie en AVR: Transmisor



# Periférico serie en AVR: Receptor



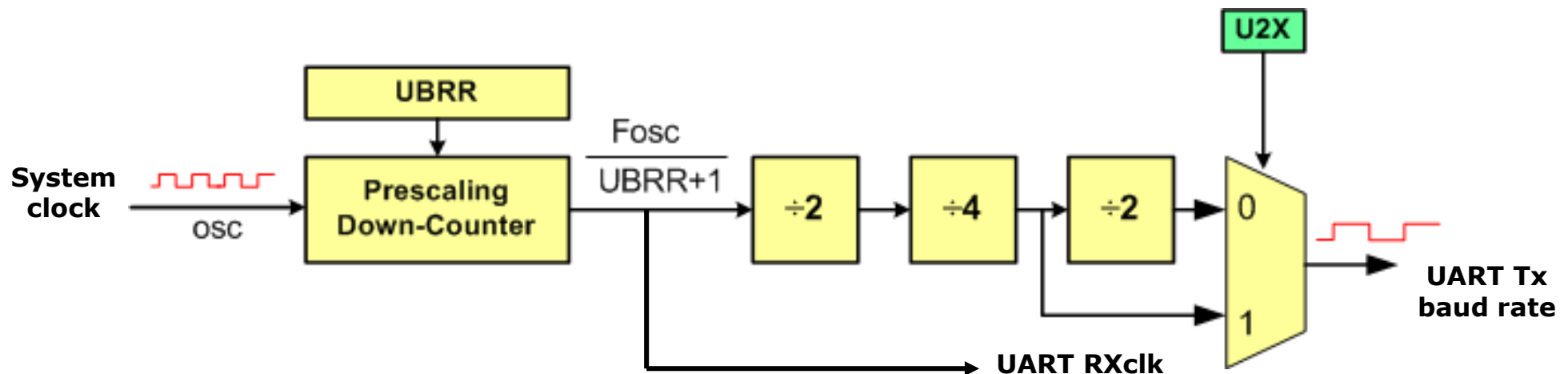
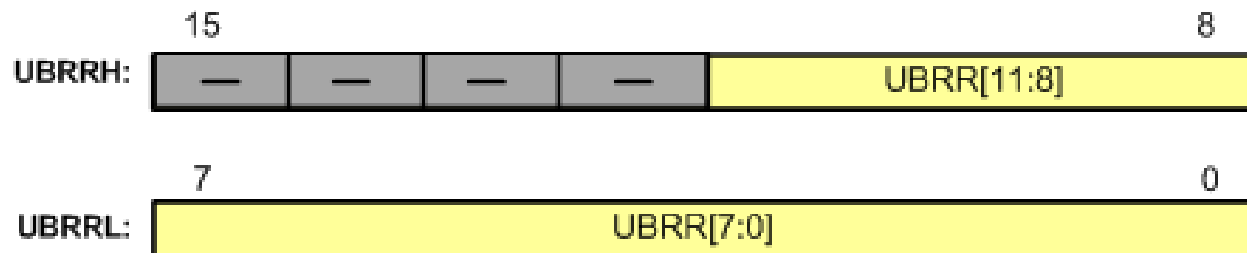
# Periférico serie en AVR: Registros de Control



# Periférico serie en AVR: Generador de reloj



$$\text{Baud rate} = \frac{F_{\text{osc}}}{(UBRR+1) \times 16} \quad \text{Normal Mode } U2x=0$$



# Periférico serie en AVR: Generador de reloj

Ejemplo: configurar UBRR para tasa de 9600bps

$$\text{Baud rate} = \frac{F_{\text{osc}}}{(\text{UBRR}+1) \times 16} \Rightarrow 9600 = \frac{16 \text{ MHz}}{(\text{UBRR}+1) \times 16}$$

$$\Rightarrow (\text{UBRR}+1) = \frac{1 \text{ MHz}}{9600} = 104.166 \Rightarrow \text{UBRR} = 103$$

$$\Rightarrow \text{Error cometido} = \{ (9600 - 1 \text{ MHz}/104) \times 100 \} / 9600 \approx 0,2\%$$

| Standard Baud Rates |
|---------------------|
| 1200                |
| 2400                |
| 4800                |
| 9600                |
| 19,200              |
| 38,400              |
| 57,600              |
| 115,200             |

¿cuál es el máximo error entre relojes que puedo tener para una comunicación confiable?

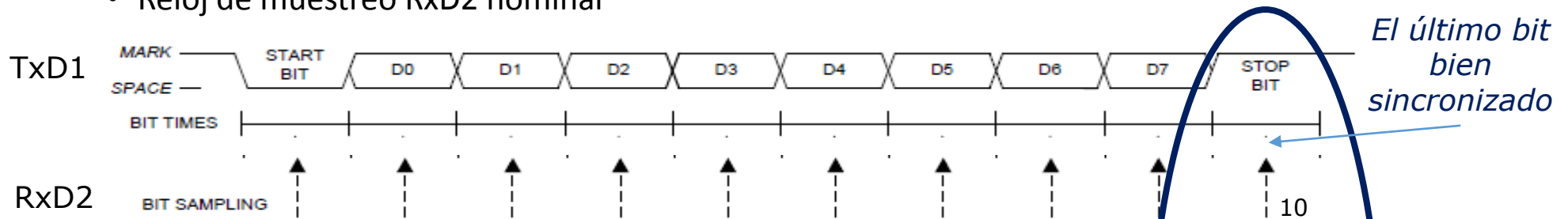


# Cota del error de tasa entre dispositivos

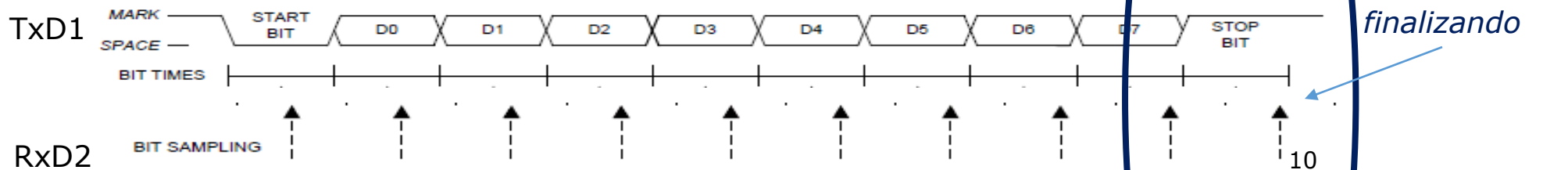
Por lo tanto:  $|\text{error}| < T_b/2$

$T_b/2$  en  $10T_b \Rightarrow 5\%$

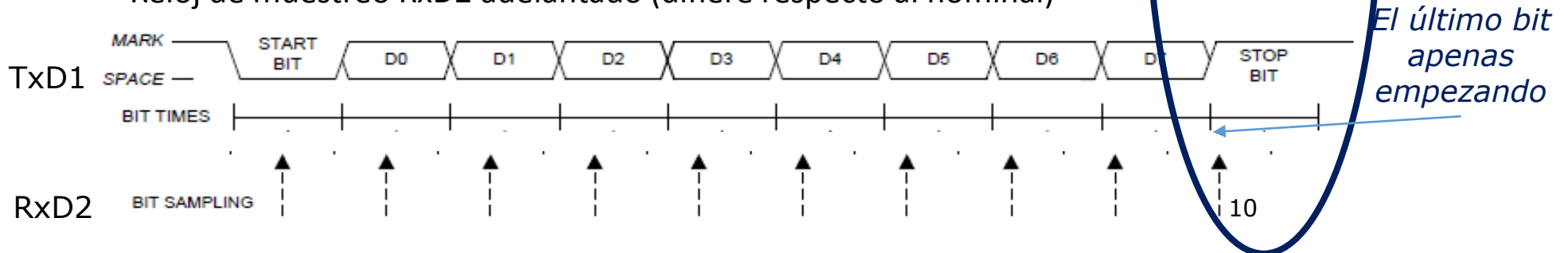
- Reloj de muestreo RxD2 nominal



- Reloj de muestreo RxD2 atrasado (difiere respecto al nominal)

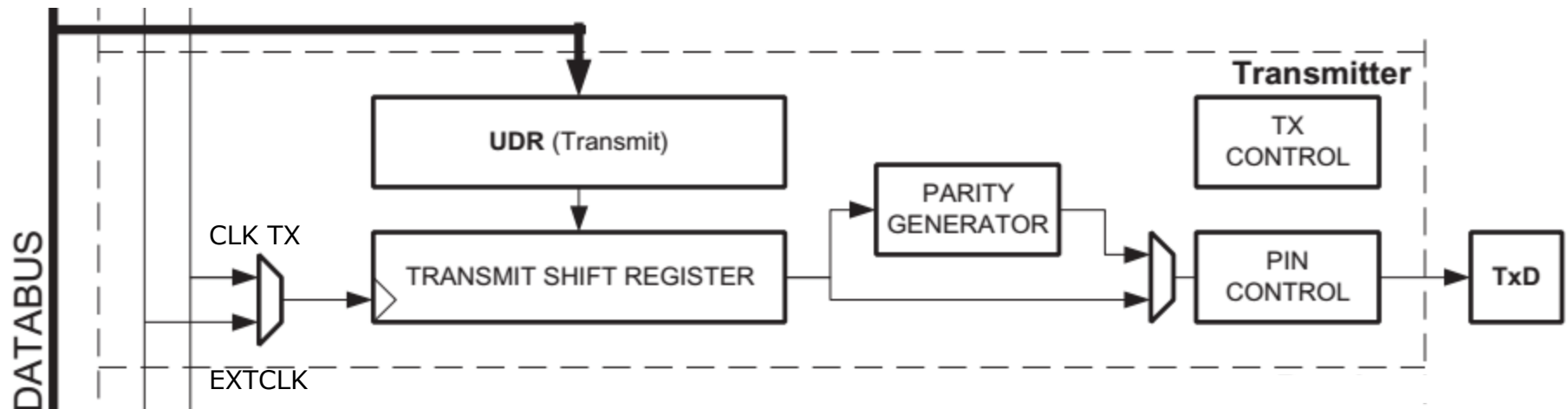


- Reloj de muestreo RxD2 adelantado (difiere respecto al nominal)



# Transmisor

Para transmitir un dato debe habilitarse el transmisor (**TXEN=1**) y configurarse apropiadamente

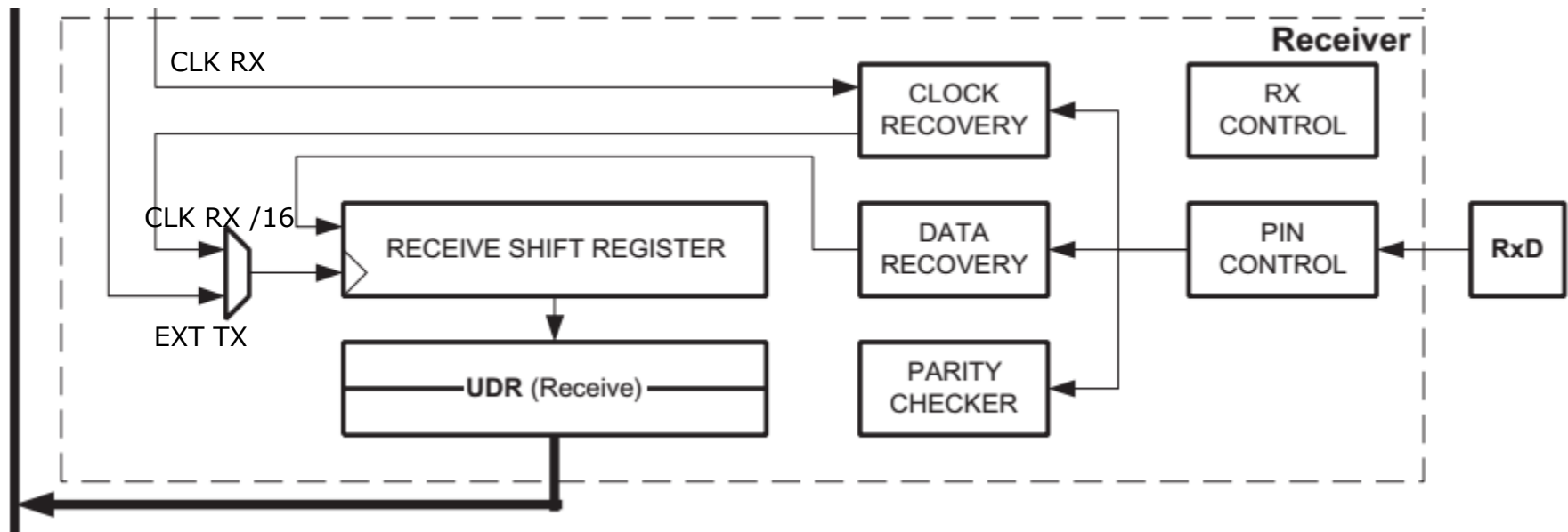


El transmisor es básicamente un conversor de datos paralelo a serie, sincronizado con un reloj derivado del reloj del microcontrolador.

- Si el **registro UDR** está vacío (flag **UDRE=1**) el usuario puede cargar un dato para transmitir. Este inmediatamente se transfiere al shift register y la transmisión comienza. El usuario puede escribir otro dato en el UDR que quedará a la espera de ser transmitido. Esto constituye un mecanismo de doble Buffer.
- El flag **TXC** se activa (**TXC=1**) cuando el transmisor haya completado la transmisión y tanto el UDR como el shift register estén vacíos.
- Ambos flag se borran automáticamente con la escritura de un dato en el UDR.

# Receptor

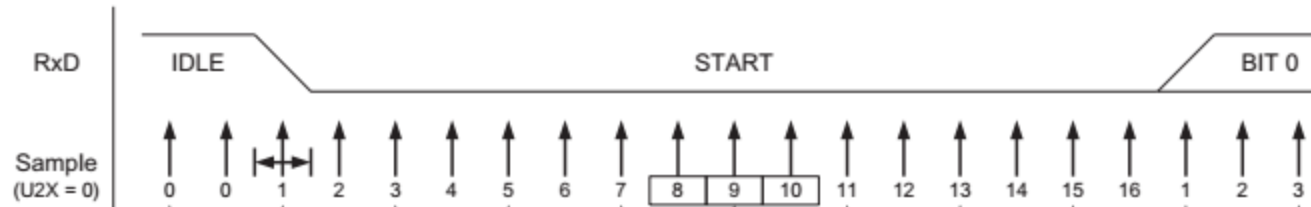
- El Receptor es básicamente un conversor de datos serie a paralelo, sincronizado con el reloj derivado del reloj del MCU (modo asincrónico) o el reloj que recibe del transmisor remoto (modo síncrono).



- Para recibir un dato debe habilitarse el receptor (**RXEN=1**) y configurarse apropiadamente

# Receptor

- Los datos presentes en el pin de entrada del periférico son muestreados a una tasa 16 veces mayor a la seleccionada de manera de detectar el bit de comienzo y sincronizarse con el centro de los bits de datos



- Una vez detectado el bit de comienzo, los datos son muestreados e introducidos al registro de desplazamiento a medida que son decodificados y hasta detectar el bit de parada o STOP
- Luego de completar la recepción el contenido del registro de desplazamiento se transfiere al **registro de datos UDR** y se activa el flag que indica “dato recibido” **RXC**. Este flag también puede generar una solicitud de interrupción si se habilita **RXCIE**.
- Luego un nuevo dato puede ser recibido, mientras el anterior se encuentra almacenado en el UDR. Esto constituye un mecanismo de doble Buffer.
- El receptor incluye además un hardware para detectar distintos tipos errores:
  - Sobrecarga de datos (flag data overrun): ocurre cuando un nuevo carácter ha sido recibido en el shift register y aún no se ha leído el dato anterior almacenado en el registro de datos
  - Error de trama (frame error): el bit de fin de trama (stop bit) no fue detectado como se esperaba.
  - Error de Paridad (parity error): el calculo de la paridad del dato recibido no coincide con el bit de paridad transmitido.

# UCSR0B: UART CONTROL AND STATUS REGISTER B (UART0)

|        |        |        |       |       |        |       |       |
|--------|--------|--------|-------|-------|--------|-------|-------|
| RXCIE0 | TXCIE0 | UDRIE0 | RXEN0 | TXEN0 | UCSZ02 | RXB80 | TXB80 |
|--------|--------|--------|-------|-------|--------|-------|-------|

- **RXCIE0** (Bit 7): Receive Complete Interrupt Enable
  - To enable the interrupt on the RXC0 flag in UCSR0A you should set this bit to one.
- **TXCIE0** (Bit 6): Transmit Complete Interrupt Enable
  - To enable the interrupt on the TXC0 flag in UCSR0A you should set this bit to one.
- **UDRIE0** (Bit 5): USART Data Register Empty Interrupt Enable
  - To enable the interrupt on the UDRE0 flag in UCSR0A you should set this bit to one.
- **RXEN0** (Bit 4): Receive Enable
  - To enable the USART receiver you should set this bit to one.
- **TXEN0** (Bit 3): Transmit Enable
  - To enable the USART transmitter you should set this bit to one.
- **UCSZ02** (Bit 2): Character Size
  - This bit combined with the UCSZ1:0 bits in UCSRC sets the number of data bits (character size) in a frame.
- **RXB80** (Bit 1): Receive data bit 8
  - This is the ninth data bit of the received character when using serial frames with nine data bits.
- **TXB80** (Bit 0): Transmit data bit 8
  - This is the ninth data bit of the transmitted character when using serial frames with nine data bits.

# UCSR0C: UART CONTROL AND STATUS REGISTER C

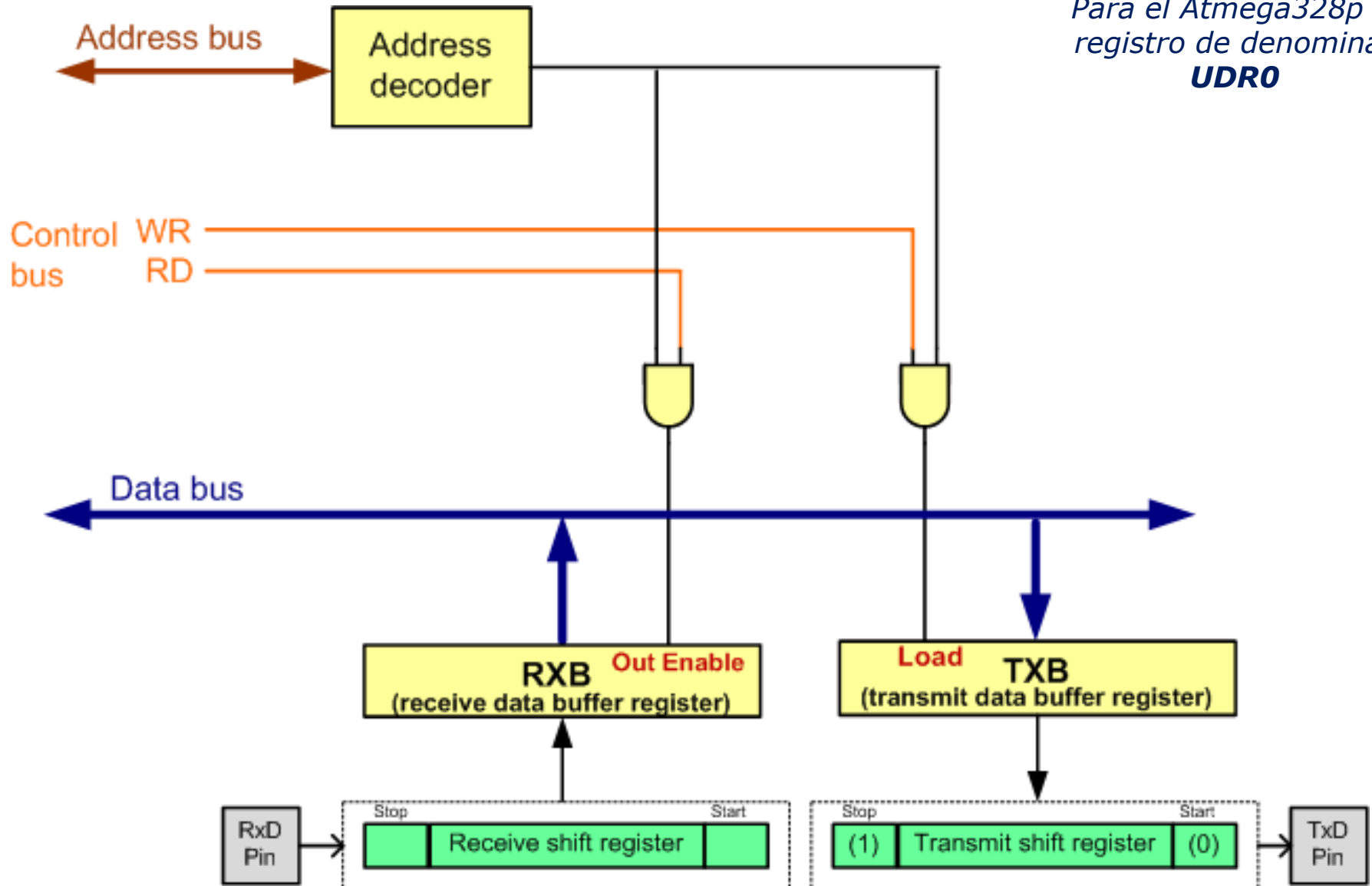
| UMSEL01 | UMSEL00 | UPM01 | UPM00 | USBS0 | UCSZ01 | UCSZ00 | UCPOL0 |
|---------|---------|-------|-------|-------|--------|--------|--------|
|---------|---------|-------|-------|-------|--------|--------|--------|

- **UMSEL01:00** (Bits 7:6): USART Mode Select
  - These bits select the operation mode of the USART:
    - 00 = Asynchronous USART operation
    - 01 = Synchronous USART operation
    - 10 = Reserved
    - 11 = Master SPI (MSPIM)
- **UPM01:00** (Bit 5:4): Parity Mode
  - These bits disable or enable and set the type of parity generation and check.
    - 00 = Disabled
    - 01 = Reserved
    - 10 = Even Parity
    - 11 = Odd Parity
- **USBS0** (Bit 3): Stop Bit Select
  - This bit selects the number of stop bits to be transmitted.
    - 0 = 1 bit
    - 1 = 2 bits
- **UCSZ01:00** (Bit 2:1): Character Size
  - These bits combined with the UCSZ02 bit in UCSR0B set the character size in a frame.
- **UCPOL0** (Bit 0): Clock Polarity
  - This bit is used for synchronous mode.

| UCSZ02 | UCSZ01 | UCSZ00 | Char. size |
|--------|--------|--------|------------|
| 0      | 0      | 0      | 5-bit      |
| 0      | 0      | 1      | 6-bit      |
| 0      | 1      | 0      | 7-bit      |
| 0      | 1      | 1      | 8-bit      |
| 1      | 1      | 1      | 9-bit      |

# UDR0: UART DATA REGISTER

*Para el Atmega328p el  
registro de denomina:  
**UDR0***



# UCSROA: UART CONTROL AND STATUS REGISTER A

|             |             |              |            |             |             |             |              |
|-------------|-------------|--------------|------------|-------------|-------------|-------------|--------------|
| <b>RXC0</b> | <b>TXC0</b> | <b>UDRE0</b> | <b>FE0</b> | <b>DOR0</b> | <b>UPE0</b> | <b>U2X0</b> | <b>MPCM0</b> |
|-------------|-------------|--------------|------------|-------------|-------------|-------------|--------------|

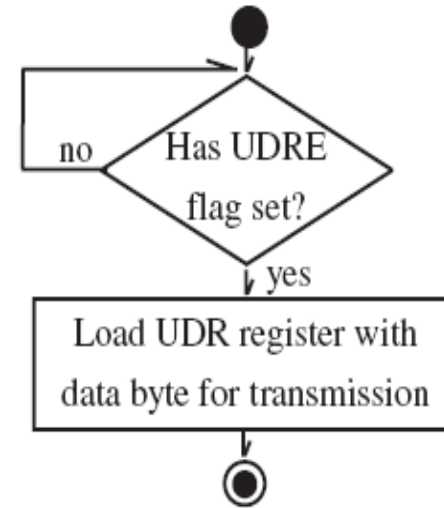
- **RXC0** (Bit 7): USART Receive Complete 0
  - This flag bit is set when there are new data in the receive buffer that are not read yet. It is cleared when the receive buffer is empty. It also can be used to generate a receive complete interrupt.
- **TXC0** (Bit 6): USART Transmit Complete 0
  - This flag bit is set when the entire frame in the transmit shift register has been transmitted and there are no new data available in the transmit data buffer register (TXB). It can be cleared by writing a one to its bit location. Also it is automatically cleared when a transmit complete interrupt is executed. It can be used to generate a transmit complete interrupt.
- **UDRE0** (Bit 5): USART Data Register Empty 0
  - This flag is set when the transmit data buffer is empty and it is ready to receive new data. If this bit is cleared you should not write to UDR0 because it overrides your last data. The UDRE0 flag can generate a data register empty interrupt.
- **FE0** (Bit 4): Frame Error 0
  - This bit is set if a frame error has occurred in receiving the next character in the receive buffer. A frame error is detected when the first stop bit of the next character in the receive buffer is zero.
- **DOR0** (Bit 3): Data OverRun 0
  - This bit is set if a data overrun is detected. A data overrun occurs when the receive data buffer and receive shift register are full, and a new start bit is detected.
- **PE0** (Bit 2): Parity Error 0
  - This bit is set if parity checking was enabled (UPM1 = 1) and the next character in the receive buffer had a parity error when received.
- **U2X0** (Bit 1): Double the USART Transmission Speed 0
- **MPCM0** (Bit 0): Multi-processor Communication Mode 0



# Ejemplo 1: enviar caracter 'G' continuamente

```
#include <avr/io.h>
```

```
int main (void) {  
    //initialize the USART  
    UCSR0B = (1<<TXEN0);  
    UCSR0C = (1<<UCSZ01)|(1<<UCSZ00);  
    UBRR0L = 103; //baud rate = 9600bps  
    while(1) { //do forever  
        while (! (UCSR0A & (1<<UDRE0))); //wait until UDR0 is empty  
        UDR0 = 'G'; //transmit 'G' letter  
    }  
    return 0;  
}
```



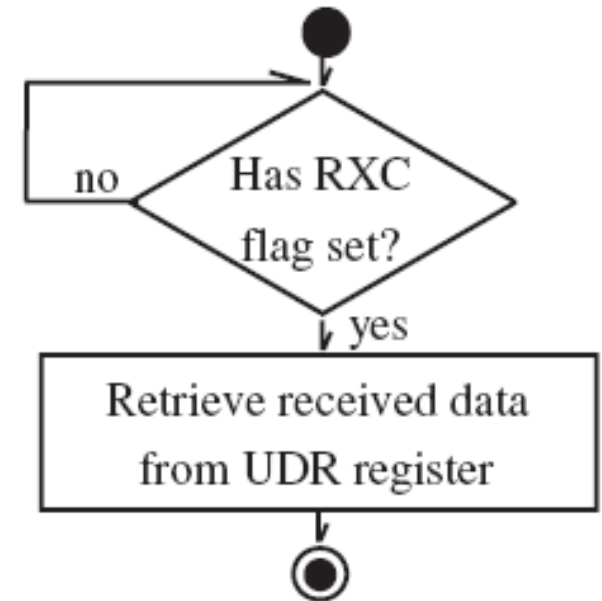
*En arduino IDE:*

```
Setup{  
    Serial.begin(9600);  
}  
Loop{  
    Serial.print('G');  
}
```

## Ejemplo 2: Recibir un dato por la interfaz serie y transferirlo al Port B.

```
#include <avr/io.h>
int main (void)
{
    DDRB = 0xFF;    //Port B is output
    UCSRB = (1<<RXEN0); //initialize USART0
    UCSRC = (1<<UCSZ01)|(1<<UCSZ00);
    UBRRL = 103;

    while(1)
    {
        while (! (UCSR0A & (1<<RXC0))); //wait until new data
        PORTB = UDR0;
    }
    return 0;
}
```



*En arduino IDE:*

```
Loop{
    if( Serial.available()){
        dato=Serial.read();
    }
}
```

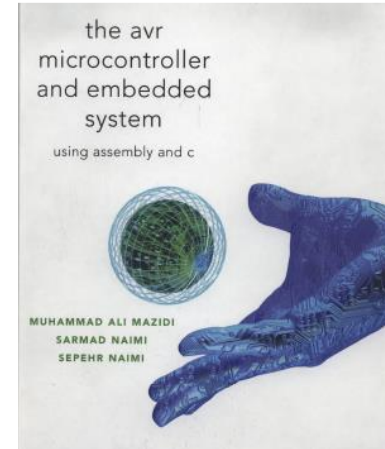
# Ejemplos para la práctica

- 1) Eco de caracteres utilizando encuesta de los flags del Tx y Rx
- 2) Eco de caracteres y uso de cadenas de caracteres ASCII.
- 3) Eco de caracteres utilizando interrupciones de Tx y Rx
- 4) Utilización de buffers de recepción, de transmisión y funciones para la comunicación con mensajes de cualquier tipo y cantidad de datos.
- 5) Creación de una biblioteca de funciones para el uso del periférico (driver con modelo productor consumidor)

# Bibliografía:

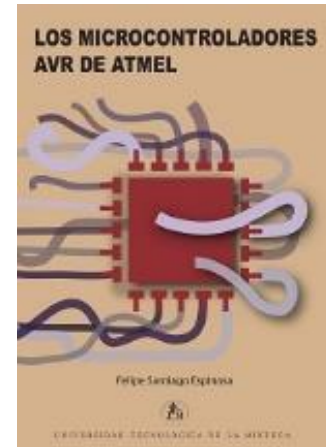
- *The AVR microcontroller & Embedded Systems.*

Mazidi, Naimis, CH11



- Libro Digital de Felipe Espinosa, CH6

- Hoja de datos ATMEGA328p, CH20



- Más info sobre implementación de comunicaciones confiables

- <https://www.luisllamas.es/arduino-caracteres-control-puerto-serie/>
- <https://www.luisllamas.es/arduino-checksum/>
- <https://www.luisllamas.es/puerto-serie-avanzado-arduino/>
- <https://www.luisllamas.es/arduino-rs485-max485/>